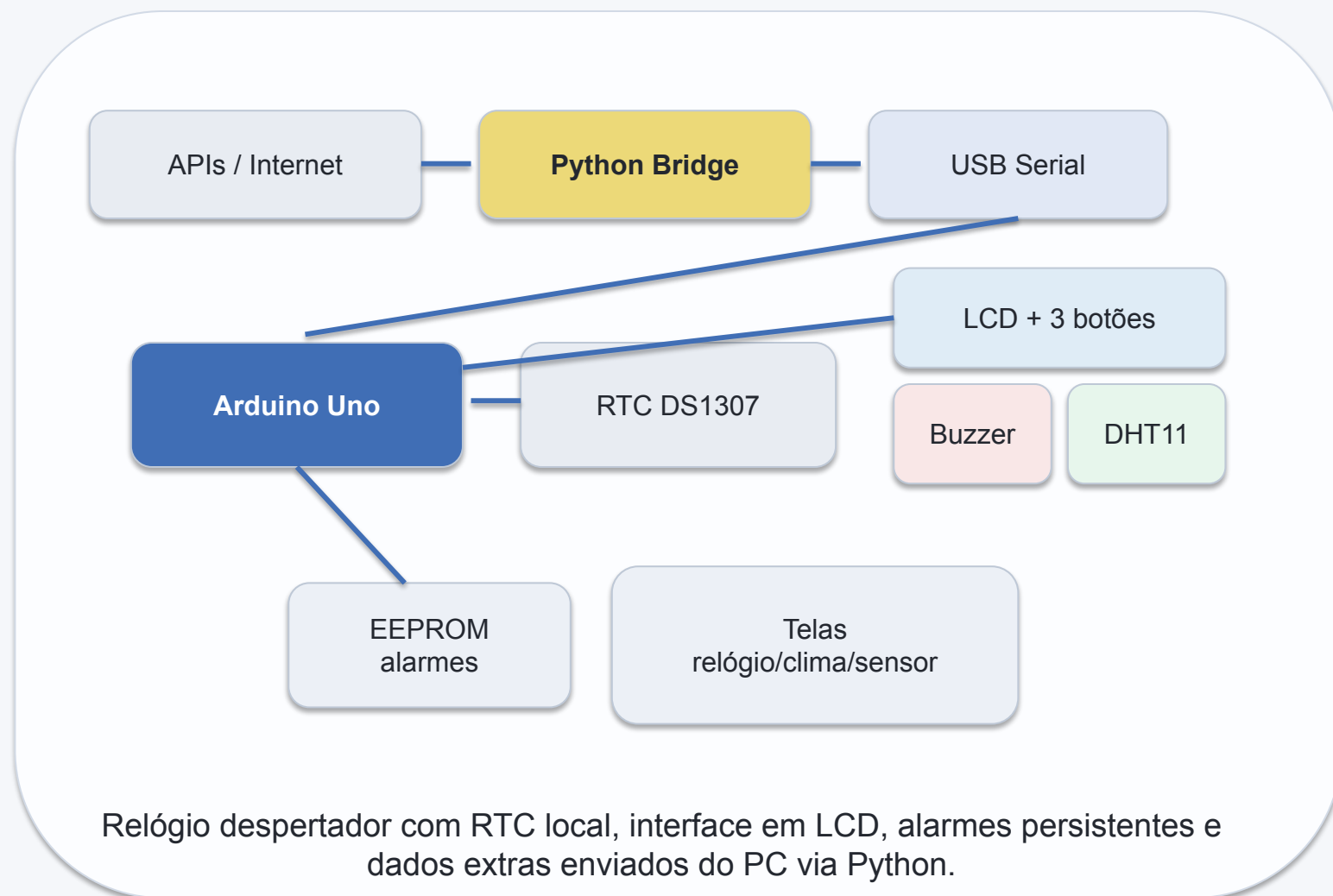


Relógio Despertador com Arduino

Arduino + Python

Guilherme Cusin Salvati
Computação Aplicada II



Projeto e objetivo

A proposta inicial era criar um relógio despertador usando o kit Arduino, com interface simples e dados extras obtidos pelo computador.

Escopo definido na proposta

Exibir data e hora em um LCD 16x2.

Permitir configurar alarmes com repetição por dias da semana.

Usar 3 botões para navegação e edição.

Adicionar telas extras com clima, precipitação e qualidade do ar.

O que foi efetivamente implementado

RTC DS1307 para manter data e hora localmente.

5 alarmes com ativação, repetição, snooze e gravação em EEPROM.

Telas de relógio, clima externo e sensor local.

Leitura de DHT11 para temperatura e umidade.

Programa em Python que envia horário e dados externos pela USB Serial.

Envio opcional de e-mail quando o alarme dispara.

Resultado: o Arduino funciona mesmo sem internet; os dados online dependem apenas do PC rodando o bridge em Python.

Arquitetura geral da solução

A lógica foi dividida entre o microcontrolador, responsável pelo funcionamento local, e um programa em Python, responsável pelos dados externos.

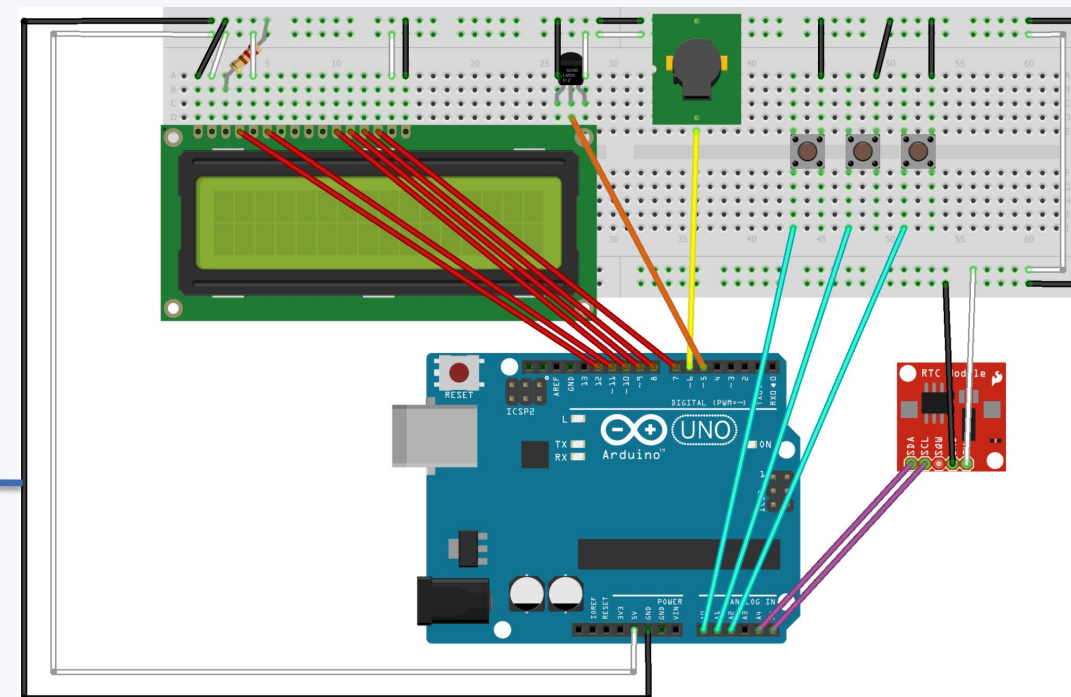
APIs externas
clima + AQI

Python bridge
pyserial + requests

Responsabilidade do Python

Consultar APIs e transformar os dados em comandos seriais simples.

Sincronizar o relógio do Arduino quando necessário.



fritzing

Responsabilidade do Arduino

Continuar operando localmente: tela, botões, alarmes, buzzer, RTC e sensor.

Hardware e pinagem

Componentes do projeto

Arduino Uno R3
LCD 16x2
RTC DS1307
3 botões de navegação
Buzzer ativo 5V
Sensor DHT11
Protoboard + jumpers + cabo USB

Mapa de portas

LCD RS/EN/D4-D7	12, 11, 10, 9, 8, 7
Buzzer	6
DHT11	5
Botão UP	A2
Botão DOWN	A0
Botão OK	A1
RTC DS1307	A4 (SDA) / A5 (SCL)

Decisões importantes

Pinos 0 e 1 ficaram reservados para a Serial USB.

Pino 13 não foi usado para evitar conflito com o LED onboard.

RTC e DHT11 garantem funcionamento local mesmo sem dados externos.

Software no Arduino

O arquivo principal do microcontrolador organiza a interface, os alarmes, o relógio, o sensor e o protocolo serial.

Interface e navegação

Máquina de estados com telas de relógio, clima externo, sensor local, menu, edição e alarme tocando.

Leitura de 3 botões com debounce e diferenciação entre toque curto e longo.

Atualização periódica do LCD sem bloquear o loop principal.

Lógica de domínio

Até 5 alarmes configuráveis.

Repetição por máscara de dias da semana.

Snooze e desligamento do alarme.

Persistência em EEPROM das configurações.

Integrações locais

RTC DS1307 para manter a data/hora.

DHT11 para temperatura e umidade locais.

Buzzer para o disparo do alarme.

Serial para receber TIME/WX e responder com ACK, STATUS, SENSOR e ALARM.

Software em Python

O programa em Python resolve a ausência de conectividade no Arduino: ele busca dados externos e os entrega ao microcontrolador pela USB Serial.

Funções principais do bridge

Detectar a porta serial automaticamente ou usar a porta informada.

Resolver a localização escolhida pelo usuário.

Consultar clima atual, precipitação e AQI.

Enviar comandos TIME e WX periodicamente ao Arduino.

Ler mensagens vindas do Arduino, como SENSOR e ALARM.

Bibliotecas e módulos

pyserial para comunicação serial.

requests para consumo das APIs.

datetime e dataclasses para o modelo de dados.

matplotlib e smtplib para geração de gráficos e envio de e-mails.

Extras presentes no repositório

Armazenamento temporário dos dados recebidos do sensor.

Geração de gráfico com histórico das medições.

Envio opcional de e-mail quando um alarme dispara.

Resultados e demonstração

A apresentação final deve mostrar o projeto funcionando localmente e a troca de dados com o computador.

Cenários de demonstração

Ajustar data e hora pelo menu do LCD.

Cadastrar um alarme com repetição semanal.

Esperar ou simular o disparo do buzzer.

Visualizar clima e AQI recebidos pela Serial.

Exibir temperatura e umidade locais no DHT11.

Situação das funcionalidades

- RTC e data/hora
- 5 alarmes
- Repetição por dias
- Snooze / stop
- Persistência em EEPROM
- Clima e AQI via Python
- Leitura do DHT11
- Envio de e-mail

Melhorias futuras

Reduzir o número de fios usando LCD com interface I2C.

Criar uma tela de configuração mais amigável para os alarmes.

Salvar preferências adicionais, como tela padrão.

Esquemática do Projeto

